

Retrieval-Augmented Generation Fundamentals

Knowledge-base document for the Domain-Specific RAG Chatbot

1. What is RAG?

Retrieval-Augmented Generation (RAG) is an approach that combines information retrieval with language-model generation. Instead of relying only on information stored in model parameters, the system retrieves relevant external passages and supplies them to the language model as context.

RAG is useful when the application needs to answer questions from a specific and changeable document collection.

2. RAG Workflow

A typical document RAG workflow is: upload documents, extract text, split text into chunks, create embeddings, store embeddings in a vector database, embed the user question, retrieve similar chunks, send the question and retrieved context to an LLM, and display the answer with source information.

The document metadata, such as filename and page number, should be preserved during ingestion so that the application can show where retrieved information came from.

3. Text Extraction

PDF documents are first processed to obtain their text. Page-level extraction is useful because it allows each extracted passage to retain the original document and page number.

Empty pages should be skipped safely. Scanned PDFs may require OCR as an optional extension when normal text extraction cannot read their content.

4. Chunking

Large documents are divided into smaller text chunks before embedding. Chunking makes retrieval more focused and prevents a single embedding from representing an excessively large amount of unrelated information.

Chunk overlap preserves some text between neighboring chunks so that an idea spanning a boundary is less likely to be separated completely.

5. Embeddings

An embedding is a numerical vector representation of text. Semantically related texts tend to have similar vector representations.

In this project, a Sentence Transformers model such as all-MiniLM-L6-v2 can convert document chunks and user questions into embeddings.

6. Vector Database and FAISS

A vector database or vector index stores embeddings and supports similarity search. FAISS is a library for efficient similarity search over dense vectors.

When a user asks a question, the question embedding is compared with stored chunk embeddings. The system can retrieve the top few most similar chunks.

7. Retrieval

Retrieval is the stage that selects evidence relevant to the user's question. A beginner RAG system can retrieve the top 3–5 chunks.

Good retrieval is essential because the language model can only produce a well-grounded answer when the relevant information is included in the supplied context.

8. Generation

The retrieved passages and the user's question are combined into a prompt and sent to a language model. The model generates an answer based on the supplied context.

A strict prompt should instruct the model to answer only from the context and to return a fallback message if the answer is not available.

9. RAG vs Fine-Tuning

RAG and fine-tuning solve different problems. RAG supplies external information at inference time, while fine-tuning changes model parameters through additional training.

RAG is often useful when the knowledge source changes frequently or when the application needs explicit document evidence and source references.

10. RAG Limitations

RAG does not automatically guarantee correct answers. If retrieval selects irrelevant chunks, the answer can still be incorrect. Chunk size, overlap, embedding quality, retrieval settings, prompt design, and model behavior can all affect results.

A RAG application should therefore be evaluated using retrieval accuracy, answer correctness, groundedness, refusal quality, source quality, and response time.